Claude Code Complete Setup Guide

From Zero to Fully Loaded — The Full Reprogramming Manual

Think of this like buying a brand-new PlayStation. Right now, it's just hardware — no games installed. This guide is every game disc, every save file, every configuration — ready to plug in and play.

What This Guide Does:
Takes a fresh VS Code + Claude Code installation and transforms it into a fully-equipped AI-powered production environment with 22+ specialized skills, 70+ knowledge files, automation tools, and battle-tested workflows.

Version 1.0 — April 2026
Built from the VS-CODING VIA CLAUDE workspace

Table of Contents

SECTION 0: WINDOWS SETUP — Read This First If You're on Windows

- Step 1: Install Node.js
- Step 2: Install Git (with Git Bash)
- Step 3: Fix the Git Bash PATH Error
- Step 4: Install Python
- Step 5: Install VS Code + Claude Code Extension
- Step 6: Verify Everything Works

Section 1: What You Need Before Starting (Prerequisites)

Section 2: The Big Picture — What You're Building

Section 3: Step-by-Step Setup

- Step 1: Create Your Project Folder
- Step 2: Build the Folder Structure
- Step 3: Set Up the .claude/ Configuration
- Step 4: Install All Skills
- Step 5: Set Up API Keys
- Step 6: Install Python Tools
- Step 7: Create Your CLAUDE.md (Brain File)
- Step 8: Set Up Knowledge Base
- Step 9: Verify Everything Works

Section 4: Complete Skills Reference (All 22+ Skills)

Section 5: Complete Folder Structure Map

Section 6: How the System Works (WAT Framework)

Section 7: Configuration Files — Exact Contents

Section 8: Troubleshooting & FAQs

Section 9: Master Checklist

SECTION 0: Windows Setup — Read This First If You're on Windows

WINDOWS USERS: START HERE

Claude Code was originally built for Mac/Linux. On Windows, it needs Git Bash to work. If you skip this section, you WILL get this error:

```
Claude Code on Windows requires Git Bash. If installed but not
in the path, set environment variable pointing to your bash.exe:
CLAUDE_CODE_GIT_BASH_PATH=C:\Program Files\Git\bin\bash.exe
```

Follow every step below IN ORDER. Do not skip any step.

Mac users — skip to Section 1.

1 Install Node.js (Required)

Claude Code runs on Node.js. You need version 18 or higher.

Download and Install:

1. Go to `nodejs.org`
2. Click the **LTS** (Long Term Support) button — this downloads the Windows installer (.msi file)
3. Run the installer
4. Click **Next** through all the screens. **IMPORTANT:** On the "Tools for Native Modules" screen, check the box that says *"Automatically install the necessary tools"* if it appears
5. Click **Install** and wait for it to finish

Verify it worked:

Open **Command Prompt** (press `Win + R`, type `cmd`, press Enter) and type:

```
node --version
```

You should see something like `v20.11.0` or higher. If you see an error, restart your computer and try again.

Also check npm (Node's package manager):

```
npm --version
```

Should show a version number like `10.2.0`.

> **If node --version shows "not recognized"**
>
> Node wasn't added to your PATH. Restart your computer first. If it still doesn't work, uninstall Node.js, re-download, and during installation make sure the "Add to PATH" checkbox is checked.

---

2 Install Git (with Git Bash) — THIS IS THE KEY STEP

This is where most people get stuck. Claude Code on Windows **requires Git Bash** — it's a Linux-like terminal that comes bundled with Git.

Download and Install:

1. Go to `git-scm.com`
2. Click **"Download for Windows"** — it will auto-detect 32-bit or 64-bit
3. Run the installer (.exe file)
4. **CRITICAL SCREENS during installation — pay attention to these:**

> **Installation Screen: "Adjusting your PATH environment"**
>
> You will see 3 options. Select the **SECOND option**:
>
> ```
> ( ) Git from Git Bash only
> (x) Git from the command line and also from 3rd-party
> software <-- SELECT THIS ONE
> ( ) Use Git and optional Unix tools from the Command Prompt
> ```
>
> This adds Git (and Git Bash) to your system PATH — which is exactly what Claude Code needs.

Installation Screen: "Choosing the default editor"

Select **"Use Visual Studio Code as Git's default editor"** if you see it. If not, any option is fine — this doesn't affect Claude Code.

Installation Screen: "Configuring the terminal emulator"

Select: **"Use Windows' default console window"**
(Either option works, but this one causes fewer issues.)

5. Click **Next** through the remaining screens (defaults are fine)
6. Click **Install** and wait for it to finish
7. **IMPORTANT: Check "Launch Git Bash"** at the end to make sure it opens

Verify it worked:

Open a **NEW** Command Prompt (close any old ones first) and type:

```
git --version
```

Should show something like `git version 2.44.0.windows.1`

Now check that bash.exe exists where Claude Code expects it:

```
dir "C:\Program Files\Git\bin\bash.exe"
```

You should see the file listed. If you see "File Not Found," try:

```
dir "C:\Program Files (x86)\Git\bin\bash.exe"
```

Write down which path worked

Whether it's `C:\Program Files\Git\bin\bash.exe` or `C:\Program Files (x86)\Git\bin\bash.exe` — you'll need this path in the next step.

3 Fix the Git Bash PATH Error — The Actual Fix

This is the error you saw. Here's how to fix it permanently:

Method A: Set the Environment Variable (Recommended — Permanent Fix)

1. Press `Win + R`, type `sysdm.cpl`, press Enter
2. Click the **"Advanced"** tab
3. Click **"Environment Variables..."** button at the bottom
4. Under **"User variables"** (top section), click **"New..."**
5. Fill in:
   - **Variable name:** `CLAUDE_CODE_GIT_BASH_PATH`
   - **Variable value:** `C:\Program Files\Git\bin\bash.exe`
6. Click **OK** on all three windows to save
7. **RESTART VS Code completely** (close it and reopen it)

Visual Step-by-Step:

```
Win + R → type "sysdm.cpl" → Enter
→ "Advanced" tab
→ "Environment Variables..." button
→ Under "User variables" → click "New..."
→ Variable name: CLAUDE_CODE_GIT_BASH_PATH
→ Variable value: C:\Program Files\Git\bin\bash.exe
→ OK → OK → OK
→ RESTART VS CODE
```

Method B: Quick Terminal Fix (Temporary — For Testing)

If you just want to test quickly before doing Method A, open Command Prompt and run:

```
set CLAUDE_CODE_GIT_BASH_PATH=C:\Program Files\Git\bin\bash.exe
```

Then launch VS Code from THAT SAME terminal:

```
code .
```

> **Method B only lasts until you close the terminal.**
>
> Always do Method A for a permanent fix. Method B is just for quick testing.

Method C: PowerShell Alternative (Also Permanent)

Open PowerShell as Administrator and run:

```
[System.Environment]::SetEnvironmentVariable(
  "CLAUDE_CODE_GIT_BASH_PATH",
  "C:\Program Files\Git\bin\bash.exe",
  "User"
)
```

Then restart VS Code.

Verify the fix worked:

Open a NEW Command Prompt and type:

```
echo %CLAUDE_CODE_GIT_BASH_PATH%
```

Should print: `C:\Program Files\Git\bin\bash.exe`

If it prints `%CLAUDE_CODE_GIT_BASH_PATH%` (the variable name itself), the variable wasn't saved. Redo Method A.

---

4 Install Python

1. Go to `python.org`
2. Click **"Download Python 3.12"** (or latest 3.x version)
3. Run the installer
4. **CRITICAL:** On the FIRST screen, check the box at the bottom that says:
   ☑ **`Add python.exe to PATH`**
   If you miss this, Python won't work from the terminal.

5. Click **"Install Now"**

Verify:

```
python --version
```

Should show `Python 3.12.x` or similar.

Also install the Python packages the tools need:

```
pip install python-pptx Pillow requests python-dotenv
```

---

5 Install VS Code + Claude Code Extension

1. Go to `code.visualstudio.com`
2. Download the **Windows** installer
3. Run the installer. During installation:
   - Check **"Add to PATH"**
   - Check **"Register Code as an editor for supported file types"**
   - Check **"Add 'Open with Code' action to Windows Explorer"** (very useful)
4. Open VS Code
5. Click the **Extensions** icon in the left sidebar (or press `Ctrl+Shift+X`)
6. Search for **"Claude Code"** by Anthropic
7. Click **Install**
8. After installation, you should see a Claude icon in the sidebar

> **Alternative: Install Claude Code via Terminal**
>
> If you prefer the CLI version, open Command Prompt and run:
>
> ```
> npm install -g @anthropic-ai/claude-code
> ```
>
> Then run `claude` in any project folder to start.

---

6 Verify EVERYTHING Works Together

Open a **NEW** Command Prompt (important — must be new to pick up PATH changes) and run ALL of these:

```
REM Check Node.js
node --version

REM Check npm
npm --version

REM Check Git
git --version

REM Check Python
python --version

REM Check pip
pip --version

REM Check Git Bash path variable
echo %CLAUDE_CODE_GIT_BASH_PATH%

REM Check that bash.exe actually exists at that path
dir "%CLAUDE_CODE_GIT_BASH_PATH%"

REM Check VS Code
code --version
```

Expected output (all should show version numbers, no errors):

```
v20.11.0                            <-- Node.js
10.2.0                              <-- npm
git version 2.44.0.windows.1       <-- Git
Python 3.12.2                       <-- Python
pip 24.0 from ...                   <-- pip
C:\Program Files\Git\bin\bash.exe   <-- Git Bash path
 bash.exe listed                    <-- File exists
1.87.0                              <-- VS Code
```

All checks pass?

Open VS Code, open the Claude Code panel, and start a conversation. The Git Bash error should be gone. You're ready to proceed to Section 1.

Still seeing the error?

Try these in order:

1. **Restart your computer**

   (not just VS Code — full restart)

2. Verify the path is correct: open File Explorer, navigate to `C:\Program Files\Git\bin\` and confirm `bash.exe` is there

3. If Git installed to a different location, find bash.exe: open Command Prompt and run
   `where bash`
   Use whatever path it shows in the environment variable

4. If `where bash` shows nothing, Git Bash didn't install properly — uninstall Git, redownload, and install again (make sure to select the SECOND PATH option in Step 2)

Windows vs Mac — Key Differences Cheat Sheet

Action	Mac (Terminal)	Windows (Command Prompt)
Open terminal	`Terminal` app	`Win + R` → `cmd` → Enter
Create folders	`mkdir -p folder/subfolder`	`mkdir folder\subfolder`
Path separator	Forward slash: `/`	Backslash: `\`
Home directory	`~/Documents/`	`%USERPROFILE%\Documents\`
List files	`ls`	`dir`
Environment variables	`export VAR=value`	`set VAR=value` (temporary) `sysdm.cpl` (permanent)
Install Python packages	`pip3 install package`	`pip install package`
Hidden folders (.claude)	Visible with `ls -a`	Show in Explorer: View → Show → Hidden items

Windows Folder Structure Command

The `mkdir -p` command in Step 2 of Section 3 is a Mac/Linux command. On Windows, use this instead (run in Command Prompt from your project folder):

```
mkdir .claude\skills\webinar
mkdir .claude\skills\ads
mkdir .claude\skills\avatar
mkdir .claude\skills\copy-master
mkdir .claude\skills\email-copy
mkdir .claude\skills\excalidraw-diagram
mkdir .claude\skills\frontend-landings\references
mkdir .claude\skills\landing-page
mkdir .claude\skills\nano-banana-images
mkdir .claude\skills\sales-page
mkdir .claude\skills\skill-builder
mkdir .claude\skills\trigger-automation
mkdir .claude\skills\ux-ui-premium-direction-style-selector
mkdir .claude\skills\video-script
mkdir .claude\skills\visualizations
mkdir .claude\skills\website-copy-extraction
mkdir .claude\settings-reference
mkdir tools
mkdir workflows\webinar-copy
mkdir 01-KNOWLEDGE\copywriting-arsenal\book-extractions
mkdir 01-KNOWLEDGE\copywriting-arsenal\swipe-analyses
mkdir 01-KNOWLEDGE\copywriting-arsenal\templates
mkdir 01-KNOWLEDGE\copywriting-arsenal\checklists
mkdir 01-KNOWLEDGE\copywriting-arsenal\anatomy-log\offer-secti
mkdir 01-KNOWLEDGE\copywriting-arsenal\anatomy-log\bonus
mkdir 01-KNOWLEDGE\copywriting-arsenal\anatomy-log\ads
mkdir 01-KNOWLEDGE\copywriting-arsenal\anatomy-log\landing-pag
mkdir 01-KNOWLEDGE\copywriting-arsenal\anatomy-log\video-scrip
mkdir 01-KNOWLEDGE\copywriting-arsenal\anatomy-log\email-copy
mkdir 01-KNOWLEDGE\copywriting-arsenal\anatomy-log\sales-page
mkdir 01-KNOWLEDGE\copywriting-arsenal\word-craft-training
mkdir 01-KNOWLEDGE\sales-page-analyses
mkdir 03-TOOLS\master-prompts
mkdir 03-TOOLS\website-copy-extraction\extractions
mkdir 03-TOOLS\website-clone-blueprints
mkdir 03-TOOLS\file-compression
mkdir webinar-library\voice
mkdir webinar-library\intro-section\proof-assets
mkdir webinar-library\content-section
mkdir webinar-library\transition-section
mkdir webinar-library\offer-section
mkdir webinar-library\objection-handling
mkdir webinar-library\avatars
mkdir webinar-library\bonuses\jay-bonus-systems\tools
mkdir brand-assets
mkdir swipe-library
```

```
mkdir images
mkdir prompts
```

Section 1: What You Need Before Starting

Before anything else, make sure you have these installed on your computer:

Required Software

Software	What It Does	How to Get It
VS Code	Your code editor — where everything happens	`code.visualstudio.com`
Claude Code Extension	The AI brain inside VS Code	VS Code Extensions → search "Claude Code" by Anthropic
Node.js (v18+)	Runs JavaScript tools and MCP servers	`nodejs.org` (LTS version)
Python (3.10+)	Runs automation scripts (PowerPoint, images, etc.)	`python.org` or via Homebrew: `brew install python`
Git	Version control (usually pre-installed on Mac)	`git-scm.com`

IMPORTANT: You Need an Anthropic Account

Claude Code requires an Anthropic API key OR a Claude Pro/Team subscription. Without this, none of the skills will work. Get it at `console.anthropic.com`.

Required API Keys

These are the "fuel" for different features. Not all are required — it depends on what you're building:

API Key	What It Powers	Required?
`ANTHROPIC_API_KEY`	Claude AI itself (if using API directly)	Only if building Claude API apps
`GEMINI_API_KEY`	Google Gemini — AI image generation	For image generation features
`KIE_API_KEY`	Kie.ai — Nano Banana 2 hyper-realistic images	For premium image generation
`IMGBB_API_KEY`	Image hosting (for reference images in AI generation)	For image workflows
`PERPLEXITY_API_KEY`	Perplexity AI — deep web research	For avatar research (Phase A)
`GMAIL_CLIENT_ID` / `SECRET`	Gmail API — email automation	For email features

Start Simple

You don't need ALL API keys to start. Claude Code itself works with just the Anthropic subscription. Add API keys as you need specific features.

Section 2: The Big Picture — What You're Building

This setup transforms Claude Code from a basic AI chat assistant into a **full production system**. Here's what you get:

The WAT Framework

Everything runs on three layers:

Layer	What It Is	Where It Lives
Workflows	Step-by-step SOPs (Standard Operating Procedures) — tells Claude WHAT to do and in what order	`workflows/`
Agent (Claude)	Claude reads workflows, runs tools, handles failures, asks when unclear	`.claude/skills/`
Tools	Python/JS scripts that do the actual work — API calls, file operations, data transforms	`tools/`

> **Why This Matters**
>
> When AI handles every step directly, accuracy drops fast. 90% accuracy per step = only 59% success after 5 steps. By offloading execution to deterministic scripts, Claude focuses on ORCHESTRATION while tools handle the precision work.

What You Get — The Full Arsenal

- **22+ Specialized Skills** — copywriting, design, landing pages, video scripts, slide decks, image generation, email sequences, and more
- **70+ Knowledge Files** — copywriting frameworks, book extractions, swipe file analyses, voice profiles
- **Python Automation Tools** — PowerPoint animation scripts, image generation, screenshot tools
- **MCP Server Integration** — Trigger.dev for workflow automation and scheduled tasks

- **Workflows** — Battle-tested SOPs for webinar copy, landing pages, slide decks, ad copy, and visual assets

Section 3: Step-by-Step Setup

How to Use This Section

Follow each step in order. Don't skip ahead. Each step builds on the previous one. Check off each item as you complete it. By the end, your Claude Code will be fully loaded.

1 Create Your Project Folder

Open Terminal and run:

```
mkdir -p ~/Documents/YOUR-PROJECT-NAME
cd ~/Documents/YOUR-PROJECT-NAME
```

Then open it in VS Code:

```
code .
```

Naming Tip

Name the folder something meaningful to your project. Example: `MY-AI-BUSINESS` or `WEBINAR-PROJECT` or `CHIDU-WORKSPACE`

---

2 Build the Folder Structure

Run this command in your Terminal to create the entire folder structure at once:

```
# Create ALL directories in one command
mkdir -p .claude/skills/webinar \
  .claude/skills/ads \
  .claude/skills/avatar \
  .claude/skills/copy-master \
  .claude/skills/email-copy \
  .claude/skills/excalidraw-diagram \
  .claude/skills/frontend-landings/references \
  .claude/skills/landing-page \
  .claude/skills/nano-banana-images \
  .claude/skills/sales-page \
  .claude/skills/skill-builder \
  .claude/skills/trigger-automation \
  .claude/skills/ux-ui-premium-direction-style-selector \
  .claude/skills/video-script \
  .claude/skills/visualizations \
  .claude/skills/website-copy-extraction \
  .claude/settings-reference \
  tools \
  workflows/webinar-copy \
  01-KNOWLEDGE/copywriting-arsenal/book-extractions \
  01-KNOWLEDGE/copywriting-arsenal/swipe-analyses \
  01-KNOWLEDGE/copywriting-arsenal/templates \
  01-KNOWLEDGE/copywriting-arsenal/checklists \
  01-KNOWLEDGE/copywriting-arsenal/anatomy-log/offer-section \
  01-KNOWLEDGE/copywriting-arsenal/anatomy-log/bonus \
  01-KNOWLEDGE/copywriting-arsenal/anatomy-log/ads \
  01-KNOWLEDGE/copywriting-arsenal/anatomy-log/landing-pages \
  01-KNOWLEDGE/copywriting-arsenal/anatomy-log/video-script \
  01-KNOWLEDGE/copywriting-arsenal/anatomy-log/email-copy \
  01-KNOWLEDGE/copywriting-arsenal/anatomy-log/sales-page \
  01-KNOWLEDGE/copywriting-arsenal/word-craft-training \
  01-KNOWLEDGE/sales-page-analyses \
  03-TOOLS/master-prompts \
  03-TOOLS/website-copy-extraction/extractions \
  03-TOOLS/website-clone-blueprints \
  03-TOOLS/file-compression \
  webinar-library/voice \
  webinar-library/intro-section/proof-assets \
  webinar-library/content-section \
  webinar-library/transition-section \
  webinar-library/offer-section \
  webinar-library/objection-handling \
  webinar-library/avatars \
  webinar-library/bonuses/jay-bonus-systems/tools \
  brand-assets \
  swipe-library \
```

```
  images \
  prompts
```

Why So Many Folders?

Each folder has a specific purpose. The skills expect files in exact locations. Think of it like game save slots — if the folder doesn't exist, the skill can't save/load from it.

3 Set Up the .claude/ Configuration

The `.claude/` folder is Claude Code's brain. Three files control everything:

File 1: `.claude/settings.json`

This controls what Claude Code is allowed to do:

```
{
  "permissions": {
    "allow": [
      "Read(~/**)"
    ]
  }
}
```

> **What This Does**
>
> Gives Claude read access to your files. Claude Code will ask permission for other operations (writing files, running scripts, etc.) the first time — you can approve them as they come up, or add more permissions here later.

File 2: `.claude/settings.local.json`

This controls which websites Claude can fetch from (for research, extraction, etc.):

```
{
  "permissions": {
    "allow": [
      "WebFetch(domain:YOUR-ALLOWED-DOMAIN-1.com)",
      "WebFetch(domain:YOUR-ALLOWED-DOMAIN-2.com)"
    ]
  }
}
```

Replace with actual domains you want Claude to access. Or leave empty and approve domains as they come up.

File 3: `.claude/mcp.json` (MCP Server Config)

This connects Claude Code to external automation platforms:

```
{
  "mcpServers": {
    "trigger.dev": {
      "command": "npx",
      "args": ["trigger.dev@4.4.0", "mcp"]
    }
  }
}
```

What's MCP?

Model Context Protocol — it lets Claude Code talk to external services. Trigger.dev is a workflow automation platform (like Zapier, but you write code). If you don't need automation, you can skip this file for now.

4 Install All Skills

Skills are `.md` files that teach Claude Code how to do specific things. Think of them as "game cartridges" — each one unlocks a new ability.

> **CRITICAL: You Need the Actual Skill Files**
>
> Skills are custom-built markdown files. They are NOT available from a public repository. You need to get them from the workspace owner (Jay/Jebril) who will share the files with you.
>
> **Ask for:** A ZIP or shared folder of the entire `.claude/skills/` directory. Copy the contents directly into your `.claude/skills/` folder.

Skills Map — What Goes Where

Flat File Skills (single .md files → drop directly into `.claude/skills/`):

File Name	What It Does
`algorithmic-art.md`	Creates animated/generative art using p5.js
`book-extraction.md`	Extracts intelligence from any book into layered reference files
`canvas-design.md`	Creates infographics, thumbnails, flyers, mockups (PNG/PDF)
`docx.md`	Creates and edits Word documents
`editing.md`	Content editing with template workflows
`frontend-design.md`	High-quality web interface design (avoids "AI look")
`pdf.md`	PDF creation, reading, merging, splitting, OCR
`pptx.md`	PowerPoint creation with speaker notes
`pptxgenjs.md`	JavaScript-based PowerPoint generation
`theme-factory.md`	10 professional design themes for slides/docs
`web-artifacts-builder.md`	Landing pages using React/Tailwind/shadcn
`webinar-slides.md`	Cinematic webinar slide decks with animations

Folder-Based Skills (each has a `SKILL.md` inside a subfolder):

Folder	Key Files Inside	What It Does
`ads/`	SKILL.md	Facebook, Instagram, YouTube, WhatsApp ad copy
`avatar/`	SKILL.md	Creates Phase B + Phase C audience research documents
`copy-master/`	SKILL.md, copy-formula-router.md	Universal copy intelligence — runs FIRST before any copy task
`email-copy/`	SKILL.md	Email & WhatsApp broadcasts and sequences
`excalidraw-diagram/`	SKILL.md	Editable Excalidraw diagram files
`frontend-landings/`	SKILL.md, references/STYLE_PRESETS.md	Premium HTML landing pages with 10 style presets
`landing-page/`	SKILL.md	Registration pages, sales pages, thank-you pages
`nano-banana-images/`	SKILL.md, gemini.md, master_prompt_reference.md	Hyper-realistic AI images via Kie.ai API
`sales-page/`	SKILL.md	Long-form sales pages and sales letters
`skill-builder/`	SKILL.md, reference.md	Creates and audits new skills

`trigger-automation/`	SKILL.md	Trigger.dev workflow automation builder
`ux-ui-premium-direction-style-selector/`	SKILL.md	8 different visual direction demos for web projects
`video-script/`	SKILL.md	Short ads (15-90s), VSL (5-20min), thank-you page videos
`visualizations/`	SKILL.md	Hand-drawn style PNG visuals
`website-copy-extraction/`	SKILL.md	Extract and analyze sales copy from any website

Webinar-Specific Skills (inside `.claude/skills/webinar/`):

File	What It Does
`orchestrator.md`	Routes ANY webinar task to the right sub-skill
`intro-skill.md`	Writes intro section (Hook → Position → Pain → Gains)
`content-skill.md`	Writes content section (Paradigm → Mechanism)
`offer-skill.md`	Writes offer section (Core Offer → Price → Scarcity → Bonuses → Guarantee)
`overlay-upgrader.md`	Upgrades any section to gold-standard quality

5 Set Up API Keys

Create a file called `.env` in your project root:

```
# ========================================
# API KEYS — Add your keys below
# ========================================

# Anthropic (Claude API — only if building API apps)
ANTHROPIC_API_KEY=your-key-here

# Google Gemini (AI image generation)
GEMINI_API_KEY=your-key-here

# Kie.ai (Nano Banana 2 — hyper-realistic images)
KIE_API_KEY=your-key-here
KIE_AI_API_KEY=your-key-here

# imgbb (Image hosting for AI generation references)
IMGBB_API_KEY=your-key-here

# Perplexity (Deep web research for avatar Phase A)
PERPLEXITY_API_KEY=your-key-here

# Gmail API (Email automation)
GMAIL_CLIENT_ID=your-id-here
GMAIL_CLIENT_SECRET=your-secret-here

# n8n/Zapier (Self-hosted automation webhooks)
N8N_WEBHOOK_URL=your-webhook-url-here
```

NEVER Commit .env to Git

Add `.env` to your `.gitignore` file. API keys are secret — if someone gets them, they can use your accounts and rack up charges.

Also create a `.env.example` file (same content but with placeholder values) so others know what keys are needed.

---

6 Install Python Tools

The `tools/` folder contains Python scripts. Install the dependencies:

```
# Install Python packages used by the tools
pip install python-pptx Pillow requests python-dotenv
```

Then copy these tool scripts from the source workspace into your `tools/` folder:

Script	What It Does
`pptx_animations.py`	Adds click-to-reveal animations to PowerPoint slides
`pptx_progressive.py`	Creates progressive reveal decks (text appears step by step)
`screenshot_proof.py`	Captures and processes screenshot proof assets
`README.md`	Documentation for all tools

Ask for the tools/ folder

Get the complete `tools/` directory from the workspace owner. These are custom Python scripts — not available via pip.

7 Create Your CLAUDE.md (The Brain File)

The `CLAUDE.md` file is the MOST IMPORTANT file in the entire setup. It lives in your project root and tells Claude Code:

- What framework to use (WAT)
- Where everything is located
- What skills are available
- Rules and behaviors to follow
- Current project status

> **Think of CLAUDE.md as the Instruction Manual**
>
> Without it, Claude Code has skills but doesn't know when or how to use them. With it, Claude Code knows EXACTLY what to do for any request.

Structure of CLAUDE.md:

```
# YOUR PROJECT NAME — Master Navigator

## WAT Framework — How to Operate
[Explains the Workflows → Agent → Tools system]

## Operating Rules
[Rules like: look for existing tools first, learn from failures,
keep workflows current, API keys in .env only]

## Skills Inventory
[Table of ALL skills with invoke triggers]

## Folder Map
[Where every file and folder lives]

## Current Project
[What you're currently working on]

## API Keys
[Reference to .env]
```

CRITICAL: Get the Actual CLAUDE.md

Ask the workspace owner for their CLAUDE.md as a starting template. Then customize it for YOUR project — update the skills inventory, folder map, current project, and any project-specific rules.

---

8 Set Up Knowledge Base (If Doing Copywriting)

The `01-KNOWLEDGE/` folder is a massive reference library. If you're doing copywriting work, this is essential:

Core Arsenal Files (16 numbered files)

#	File	Contains
00	MASTER-SYSTEM-PROMPT.md	Arsenal system prompt
01	core-frameworks.md	25 copywriting frameworks (AIDA, PAS, PASTOR, etc.)
02	awareness-and-sophistication.md	Schwartz 5 levels, sophistication stages
03	psychological-triggers.md	Cialdini's triggers, tribal identity, arousal science
04	pre-suasion-and-priming.md	Mystery hooks, open loops, Zeigarnik effect
05	master-swipe-patterns.md	Classic ad breakdowns (Halbert, Hopkins, Ogilvy)
06	positioning-strategy.md	Ries/Trout positioning, Onliness Statement
07	story-frameworks.md	12+ story frameworks for copy
08	voice-profiles.md	13 legendary copywriter voice profiles
09	adaptation-protocol.md	Media-specific adaptation rules
10	neuromarketing-and-buying-psychology.md	VAKOG, somatic markers, mirror neurons
11	market-specific-guides.md	Nigerian market, WhatsApp dynamics
12	differentiation-and-category-design.md	USP, ZAG, Purple Cow, category creation
13	virality-and-social-transmission.md	SUCCESs stickiness + STEPPS virality
14	creative-ideation-and-big-ideas.md	Finding the Big Idea
15	slippery-slide-and-readership.md	Sugarman's 31 triggers, engagement mechanics

Book Extractions (22 files)

Full intelligence extractions from legendary copywriting and marketing books:

- Breakthrough Advertising (Schwartz)

- Influence Parts 1 & 2 (Cialdini)
- Pre-Suasion (Cialdini)
- Adweek Copywriting Handbook (Sugarman)
- The Boron Letters (Halbert)
- Ogilvy on Advertising
- Scientific Advertising (Hopkins)
- Positioning (Ries & Trout)
- Made to Stick (Heath Brothers)
- Contagious (Berger)
- Purple Cow (Godin)
- Cashvertising (Whitman)
- Hey Whipple (Sullivan)
- Great Leads (Masterson)
- ...and more

Swipe File Analyses (16 files)

Detailed breakdowns of legendary ads: Halbert's dollar letter, Hopkins' Schlitz beer, Ogilvy's Hathaway shirt, Schwartz's boardroom reports, Sugarman's BluBlocker, and more.

Word Craft Training (3 files)

- `jason-word-craft-study-batch1.md` — 19 word craft techniques
- `jason-word-craft-study-batch2.md` — Advanced techniques
- `fang-yuan-compressed-wisdom-study.md` — Compression and precision

> **Not Doing Copywriting?**
>
> If your project is purely web development, design, or automation — you can skip the 01-KNOWLEDGE folder entirely. The skills still work without it. The knowledge base just makes the copy-related skills dramatically more powerful.

9 Verify Everything Works

Open Claude Code in VS Code and test each layer:

Test 1: Basic Communication

Type in Claude Code chat: *"What skills do you have available?"*

Claude should list all the skills from your CLAUDE.md file.

Test 2: Skill Invocation

Try invoking a simple skill: *"Create a simple PDF document with the title 'Test'"*

If the PDF skill is installed correctly, Claude will generate a PDF.

Test 3: Folder Awareness

Ask: *"Show me the folder structure of this project"*

Claude should navigate and describe your folder structure correctly.

Test 4: Tool Execution

If you have Python tools installed, ask Claude to run a test:

```
python tools/test_animation.py
```

Test 5: API Keys

Ask Claude to check if API keys are configured: *"Check if my .env file has the required API keys"*

> **All Tests Pass?**
>
> You're fully loaded. Start building.

Section 4: Complete Skills Reference

Here's every skill, what triggers it, and what it produces:

Design & Visual Skills

Skill	Trigger Phrases	Output
canvas-design	"create an infographic," "make a thumbnail," "design a flyer"	PNG or PDF visual
algorithmic-art	"create generative art," "make animated visuals"	p5.js animation
excalidraw-diagram	"draw a diagram," "make a flowchart"	.excalidraw file
visualizations	"draw a visual," "make a hand-drawn visual"	PNG image
nano-banana-images	"generate a realistic image," "create a photo"	AI-generated image via Kie.ai
theme-factory	"create a theme," "design a color palette"	10 theme presets

Web & Frontend Skills

Skill	Trigger Phrases	Output
frontend-design	"build a web component," "create a page"	Production-grade HTML/CSS/JS
frontend-landings	"create a landing page," "build a marketing page"	Single-file animated HTML
landing-page	"build a registration page," "create a thank-you page"	Full landing page
web-artifacts-builder	"build a React component"	React/Tailwind/shadcn artifacts
ux-ui-premium-direction	"show me design directions," "compare visual styles"	8 different design demos

Copywriting & Marketing Skills

Skill	Trigger Phrases	Output
copy-master	"write copy for," "create an ad," (runs FIRST automatically)	Copy intelligence brief
ads	"write a Facebook ad," "create Instagram copy"	3 ad variations with hooks
sales-page	"write a sales page," "create a sales letter"	Long-form sales copy
email-copy	"write an email," "create a launch sequence"	Email/WhatsApp copy
video-script	"write a video ad," "write a VSL"	Video scripts (short/long)
avatar	"create avatar," "build Phase B," "research audience"	Phase B + C audience docs
website-copy-extraction	"analyze this sales page," "extract copy from"	Copy extraction analysis

Document & Presentation Skills

Skill	Trigger Phrases	Output
pdf	"create a PDF," "merge PDFs," "extract from PDF"	PDF files
pptx	"create a presentation," "make slides"	PowerPoint files
docx	"create a Word document"	Word documents
webinar-slides	"create webinar slides," "build slide deck"	Cinematic webinar PPTX
book-extraction	"extract [book]," "add [book] to arsenal"	5-layer intelligence file

Automation & Utility Skills

Skill	Trigger Phrases	Output
trigger-automation	"create an automation," "set up a webhook"	Trigger.dev TypeScript tasks
skill-builder	"create a new skill," "audit this skill"	New or optimized skill files

Section 5: Complete Folder Structure Map

```
YOUR-PROJECT/
├── CLAUDE.md                          ← THE BRAIN FILE (most important)
├── .env                               ← API keys (NEVER commit to git)
├── .env.example                       ← Template showing which keys are needed
├── .gitignore                         ← Must include .env
│
├── .claude/                           ← Claude Code configuration
│   ├── mcp.json                       ← MCP server connections
│   ├── settings.json                  ← Global permissions
│   ├── settings.local.json            ← Local overrides (web fetch domains)
│   ├── settings-reference/            ← Reference configs
│   │   └── frontend-rules.md
│   └── skills/                        ← ALL SKILLS LIVE HERE
│       ├── algorithmic-art.md         ← Flat file skills
│       ├── book-extraction.md
│       ├── canvas-design.md
│       ├── docx.md
│       ├── editing.md
│       ├── frontend-design.md
│       ├── pdf.md
│       ├── pptx.md
│       ├── pptxgenjs.md
│       ├── theme-factory.md
│       ├── web-artifacts-builder.md
│       ├── webinar-slides.md
│       ├── ads/                       ← Folder-based skills
│       │   └── SKILL.md
│       ├── avatar/
│       │   └── SKILL.md
│       ├── copy-master/
│       │   ├── SKILL.md
│       │   └── copy-formula-router.md
│       ├── email-copy/
│       │   └── SKILL.md
│       ├── excalidraw-diagram/
│       │   └── SKILL.md
│       ├── frontend-landings/
│       │   ├── SKILL.md
│       │   └── references/
│       │       └── STYLE_PRESETS.md
│       ├── landing-page/
│       │   └── SKILL.md
│       ├── nano-banana-images/
│       │   ├── SKILL.md
│       │   ├── gemini.md
│       │   └── master_prompt_reference.md
```

```
│       ├── sales-page/
│       │   └── SKILL.md
│       ├── skill-builder/
│       │   ├── SKILL.md
│       │   └── reference.md
│       ├── trigger-automation/
│       │   └── SKILL.md
│       ├── video-script/
│       │   └── SKILL.md
│       ├── visualizations/
│       │   └── SKILL.md
│       ├── website-copy-extraction/
│       │   └── SKILL.md
│       └── webinar/                ← Webinar-specific skills
│           ├── orchestrator.md
│           ├── intro-skill.md
│           ├── content-skill.md
│           ├── offer-skill.md
│           └── overlay-upgrader.md
│
├── tools/                          ← Python automation scripts
│   ├── pptx_animations.py
│   ├── pptx_progressive.py
│   ├── screenshot_proof.py
│   └── README.md
│
├── workflows/                      ← Standard Operating Procedures
│   ├── webinar-landing-page.md
│   ├── webinar-ad-copy.md
│   ├── webinar-visual-assets.md
│   ├── webinar-slides.md
│   └── webinar-copy/               ← All webinar copy outputs
│
├── 01-KNOWLEDGE/                   ← Reference library (read-only)
│   ├── copywriting-arsenal/        ← 16 core files + extractions
│   │   ├── 00 through 15 numbered files
│   │   ├── book-extractions/       ← 22 book extractions
│   │   ├── swipe-analyses/         ← 16 ad breakdowns
│   │   ├── templates/              ← Headlines + sales letter templates
│   │   ├── word-craft-training/    ← Fang Yuan + Jason word craft
│   │   └── anatomy-log/            ← Copy anatomy by type
│   └── sales-page-analyses/
│
├── 03-TOOLS/                       ← Reusable prompts & templates
│   ├── master-prompts/
│   ├── website-clone-blueprints/
│   ├── website-copy-extraction/
│   └── file-compression/
│
├── webinar-library/                ← Webinar reference materials
```

```
│   ├── voice/                    ← Voice foundation
│   ├── intro-section/            ← Overlays + craft guides
│   ├── content-section/
│   ├── offer-section/            ← 9 files (GOLD STANDARD)
│   ├── avatars/                  ← Phase B + C per project
│   └── bonuses/                  ← Bonus system definitions
│
├── brand-assets/                 ← Logo, colors, typography
├── images/                       ← AI-generated images output
├── prompts/                      ← Image generation prompts log
└── swipe-library/                ← Collected swipe files
```

Section 6: How the System Works

The WAT Framework in Practice

Here's how a real task flows through the system:

Example: "Create a Facebook ad for my webinar"

1. **Claude reads CLAUDE.md**

   → sees that ad copy tasks should use the `ads/` skill

2. **Claude checks if copy-master should run first**

   → yes, for any copy task, copy-master runs first (12-step intelligence system)

3. **copy-master produces a brief**

   → identifies offer, researches market, diagnoses awareness level, selects emotional arc and framework

4. **ads skill takes the brief**

   → generates 3 ad variations with different hook types

5. **If the ad needs an image**

   → Claude routes to `nano-banana-images/` or `canvas-design`

6. **Result:**

   Complete ad package — copy + visuals — ready to post

Key Operating Rules

1. **Look for existing tools first.** Check `tools/` before building anything new.
2. **Learn and adapt when things fail.** Read error, fix script, retest. Document what you learned.
3. **Keep workflows current.** When you find better methods, update the workflow.
4. **Self-improvement loop:** Identify break → fix tool → verify → update workflow → move on stronger.
5. **API keys in .env only.** Never hardcode. Never log secret values.

Skill Routing — How Claude Decides What To Use

CLAUDE.md contains an "Invoke When" column for every skill. Claude matches your request to the right skill automatically. Examples:

- "Write a sales page" → `copy-master` (first) → `sales-page`
- "Create webinar slides" → `webinar-slides`
- "Build a landing page" → `landing-page`
- "Generate a realistic photo" → `nano-banana-images`
- "Extract intelligence from this book" → `book-extraction`
- "Set up a cron job" → `trigger-automation`

Section 7: Configuration Files — What Each One Does

File	Location	Purpose	Required?
`CLAUDE.md`	Project root	Master instruction file — tells Claude everything about the project	**YES — #1 priority**
`.env`	Project root	API keys and secrets	YES (for any API-powered features)
`.claude/settings.json`	.claude/	Global Claude Code permissions	YES
`.claude/settings.local.json`	.claude/	Local overrides (allowed websites, etc.)	Optional
`.claude/mcp.json`	.claude/	External service connections (Trigger.dev, etc.)	Only for automation
`.gitignore`	Project root	Prevents .env and other secrets from being committed	YES
`trigger-ref.md`	Project root	Trigger.dev SDK v4 code reference	Only for automation

Section 8: Troubleshooting & FAQs

Q: Claude doesn't know about my skills

Fix: Check that CLAUDE.md exists in your project root AND lists the skills. Claude reads CLAUDE.md on every conversation start.

Q: A skill says "file not found"

Fix: The skill expects files in specific folder paths. Make sure your folder structure matches Section 5 exactly. Run the mkdir command from Step 2 again.

Q: API calls are failing

Fix: Check your .env file. Make sure the key names match exactly (no extra spaces). Test the key manually if possible.

Q: Python tools won't run

Fix: Run `pip install python-pptx Pillow requests python-dotenv`. Make sure Python 3.10+ is installed: `python3 --version`

Q: MCP server won't connect

Fix: Make sure Node.js is installed (`node --version`). The MCP config uses npx which requires Node. Restart VS Code after adding mcp.json.

Q: Claude isn't following the rules I set

Fix: Rules go in CLAUDE.md. Make sure they're clear, specific, and not contradicting each other. Claude re-reads CLAUDE.md each conversation.

Q: I want to add a new skill

How: Use the `skill-builder` skill. Tell Claude: "Create a new skill for [what you need]." It will generate the SKILL.md with proper structure.

Q: How do I update an existing skill?

How: Edit the SKILL.md file directly, or use skill-builder to audit and optimize it.

Section 9: Master Checklist

Print this page. Check off each item as you complete it.

Phase 1: Foundation

- ☐ VS Code installed and working
- ☐ Claude Code extension installed from VS Code marketplace
- ☐ Node.js v18+ installed (`node --version` works)
- ☐ Python 3.10+ installed (`python3 --version` works)
- ☐ Git installed (`git --version` works)
- ☐ Anthropic account active (API key or Claude Pro subscription)

Phase 2: Project Structure

- ☐ Project folder created
- ☐ Full folder structure created (Step 2 mkdir command)
- ☐ `.gitignore` created (includes `.env`)

Phase 3: Configuration

- ☐ `.claude/settings.json` created with read permissions
- ☐ `.claude/settings.local.json` created (even if empty)
- ☐ `.claude/mcp.json` created (if using automation)
- ☐ `.env` file created with required API keys
- ☐ `.env.example` created as template

Phase 4: Skills Installation

- ☐ All 12 flat-file skills copied to `.claude/skills/`
- ☐ All 15 folder-based skills copied with their SKILL.md files
- ☐ Webinar skills (5 files) copied to `.claude/skills/webinar/`
- ☐ Supporting files copied (copy-formula-router.md, STYLE_PRESETS.md, gemini.md, etc.)

Phase 5: Tools & Dependencies

- ☐ Python packages installed (`pip install python-pptx Pillow requests python-dotenv`)
- ☐ Tool scripts copied to `tools/` folder

Phase 6: CLAUDE.md (Brain File)

- ☐ CLAUDE.md created in project root
- ☐ Skills inventory section populated
- ☐ Folder map section populated
- ☐ Operating rules defined
- ☐ Current project section filled in

Phase 7: Knowledge Base (Copywriting Only)

- ☐ 16 core arsenal files in `01-KNOWLEDGE/copywriting-arsenal/`
- ☐ Book extractions copied to `book-extractions/`
- ☐ Swipe analyses copied to `swipe-analyses/`
- ☐ Word craft training files copied
- ☐ Templates and checklists copied

Phase 8: Verification

- ☐ Claude recognizes skills when asked
- ☐ At least one skill runs successfully
- ☐ Folder structure is correct
- ☐ API keys are accessible (if configured)
- ☐ Python tools execute without errors

ALL BOXES CHECKED? YOU'RE FULLY LOADED.

Your Claude Code is now a production-ready AI workspace.
Start building. Start creating. Start shipping.

Claude Code Complete Setup Guide — Version 1.0 — April 2026

Built from the VS-CODING VIA CLAUDE workspace